

Student Portal: Microservices, Docker, Database, and Frontend Guide

Beginner-friendly short guide for running and understanding the project

Fast Mental Model

One frontend talks to many Spring Boot REST microservices. Each service owns a focused job and stores data in MySQL. Docker starts everything together.

Core Ports

Service	Port	Purpose
frontend	8080	Nginx static frontend
mysql	3306	MySQL database
Portal_Service	9110	Attendance and in-app mail
FacultyRegister	9070	Faculty login
FacultyHome	9071	Faculty dashboard/courses
Admin_Students	9098	Admin student data
Admin_Faculty	9102	Admin faculty data
AdminDashboard	9083	Dashboard counts

1. What This Project Contains

- Frontend: HTML, CSS, and JavaScript pages inside frontend/.
- Backend: multiple Spring Boot REST microservices inside backend/Admin/.
- Database: MySQL databases used by each service. Hibernate creates/updates tables from entities.
- Docker: docker-compose.yml runs MySQL, all backend services, and an Nginx frontend server.
- Portal_Service: new service for student attendance summaries and internal in-app mail.

2. Simple Architecture Flow

- Admin creates master data: students, faculty, courses, timetable, fee details.
- Faculty logs in and sees courses assigned by admin.
- Faculty marks attendance by section and subject.
- Portal_Service stores attendance records in MySQL.
- Student opens Attendance and sees only their own subject-wise percentage.
- Mail is internal application mail only: sender chooses recipient role, name, and ID. Only that recipient can see the message inside the application.

3. Microservices Used

- Admin_Register - admin login/register service on port 9093.
- Admin_Students - stores students created by admin on port 9098.
- Admin_Faculty - stores faculty created by admin on port 9102.
- Admin_Student_Course - stores student course data on port 9094.
- Admin_Faculty_Course - stores faculty course mapping on port 9095.
- Admin_Students_Timetable - student timetable service on port 9100.
- Admin_Faculty_Timetable - faculty timetable service on port 9099.
- AdminDashboard - aggregates counts from students/faculty services on port 9083.

- FacultyRegister - faculty login service on port 9070.
- FacultyHome - faculty course dashboard service on port 9071.
- MyFee - fee service on port 9096.
- Portal_Service - attendance and internal mail service on port 9110.

4. Important New APIs

- POST /portal/attendance/mark - faculty submits attendance for a subject/date/section.
- GET /portal/attendance/student/{registerNumber}/summary - student sees subject-wise attendance.
- POST /portal/mail/send - sends mail inside the application only.
- GET /portal/mail/inbox?role=STUDENT&id=REGNO - loads messages for one exact recipient.

5. How Docker Combines Everything

- docker-compose.yml starts one MySQL container first.
- The MySQL init script creates all required databases automatically.
- Each Spring Boot service receives its database URL using SPRING_DATASOURCE_URL.
- Backend services communicate by Docker service name, for example http://admin-students:9098.
- The frontend runs through Nginx on port 8080 and the browser calls localhost backend ports.

6. Run With Docker

- Install Docker Desktop and keep it running.
- Open PowerShell in C:\Htmlprojects\Student_Portal.
- Run: docker compose up --build
- Open frontend: http://localhost:8080/FirstPage.html
- Stop all containers: press Ctrl+C, then run docker compose down.

7. Run Without Docker

- Start MySQL locally.
- Create databases from docker/mysql/init/01-create-databases.sql or database/*.sql.
- Run start-all.bat to start all Spring Boot services in separate windows.
- Open frontend/FirstPage.html directly in the browser, or serve frontend/ with a small static server.

8. Database Notes

- For Docker, no manual database creation is needed. The init SQL creates databases.
- For local MySQL, create the same databases before starting services.
- Hibernate ddl-auto=update lets Spring Boot create/update tables from entity classes.
- Database passwords are not hard-coded now. They come from environment variables.

9. Frontend Responsiveness Work

- The existing design was kept: same colors, sidebar, buttons, cards, and page structure.
- Fixed fixed-margin sidebar label alignment by allowing text and icons to flex naturally.
- Added mobile/tablet breakpoints for forms, tables, mail, attendance grid, and cards.
- Tables scroll horizontally on small screens instead of breaking layout.
- Added smooth hover/focus transitions without changing the visual theme.

Simple Code Examples

Docker command

```
cd C:\Htmlprojects\Student_Portal
docker compose up --build
open http://localhost:8080/FirstPage.html
```

Attendance request example

```
POST http://localhost:9110/portal/attendance/mark
{
  "courseName": "DevOps & Microservices",
  "section": "MCA",
  "facultyName": "Faculty Name",
  "facultyRegisterNumber": "FAC001",
  "attendanceDate": "2026-07-17",
  "students": [
    {"studentName": "Student One", "studentRegisterNumber": "MCA001", "status": "PRESENT"}
  ]
}
```

Internal mail request example

```
POST http://localhost:9110/portal/mail/send
{
  "senderRole": "FACULTY",
  "senderName": "Faculty Name",
  "senderId": "FAC001",
  "recipientRole": "STUDENT",
  "recipientName": "Student One",
  "recipientId": "MCA001",
  "subject": "Class update",
  "body": "Tomorrow class starts at 9 AM."
}
```

Learning Path

1. Understand the attendance flow from faculty to student.
2. Learn how JSON moves through REST endpoints.
3. Learn Hibernate entities and database tables.
4. Learn Docker Compose by reading one service block at a time.